全栈开发技术期末项目报告

阅记

阅读与书摘管理应用

项目名称：阅记阅读与书摘管理应用

姓名：周宇斌

学号：20246085

班级：软件 2401

2026 年 10 月 21 日

目录

项目背景与意义 2
1 需求分析 2
1.1 功能需求 2
1.2 非功能需求 3
2 系统设计 3
2.1 架构设计 3
2.2 功能模块设计 4
2.3 数据/数据库设计 4
2.4 接口与类设计 5
2.5 技术选型说明 5
3 系统实现 6
3.1 关键模块的核心实现 6
3.2 核心功能与 UI 截图 8
3.3 难点和解决方案 15
4 系统测试 16
4.1 测试环境 16
4.2 测试用例 17
4.3 测试截图 18
5 总结与展望 22

项目背景与意义

个人阅读中，书籍资料、阅读活动与摘录通常分散记录。阅记以本地离线应用统一管理这三类信息，并由阅读记录推导当前进度、计划状态和统计结果，使用户能够追踪阅读活动、整理感想及恢复自己的数据。

项目不提供电子书阅读器、账号登录、联网检索或云同步。其业务边界是“记录阅读”，用户录入的阅读记录表示自身活动声明，不能据此推断所有未记录页段均已读过。演示数据中的书名、作者、摘录和感想均明确标注为虚构资料。

1 需求分析

1.1 功能需求

应用实现录入书籍、制定计划、记录阅读、整理书摘和查看统计的完整闭环。十个独立业务页面及主要能力如下。

页面	已实现能力
阅读概览	在读书籍、近期记录、待完成计划、全量累计指标、书库与详情入口。
我的书库	书名或作者搜索、未开始/在读/已完成筛选、每页 10 本分页、新增及详情入口。
书籍详情	资料、当前进度、累计活动、记录增改删弹窗、关联书摘与计划、级联删除确认。
书籍编辑	书名、作者、分类、总页数与简介编辑，校验已有最大引用页码。
书摘列表	正文或书名搜索、收藏筛选、编辑、收藏切换和删除。
书摘编辑	选择书籍、编辑摘录与感想、可选页码和收藏状态。
阅读计划	查看进行中、已完成与已逾期计划，编辑和删除。
计划编辑	选择书籍、设定目标日期、根据剩余页数和天数展示建议。
阅读统计	累计页数、分钟数、完成书籍数与近七日阅读趋势。
设置与数据	浅深主题、空库载入虚构演示数据、JSON 导出与带自动备份的恢复。

初始化页、主窗口、主容器和阅读记录弹窗不用于补足十个业务页面。编辑未保存时，返回、切换菜单与关闭窗口共用保存、放弃、取消离开语义；保存失败保留草稿。列表返回刷新数据，保留当前搜索条件。

1.2 非功能需求

业务数据保存在用户本地应用数据目录，断网后核心业务可用。首次依赖还原需要 NuGet 来源；“运行离线”不等于“首次还原离线”。数据库写入串行，跨表修改采用事务。异步命令处理 IO、忙碌状态和错误重试。

界面使用中文标签、侧边导航、统一表单及浅深主题。原生文件选择器使用系统语言，Linux 历史测试中的 GTK 选择器部分文字为英文；Windows 本机使用实际系统文件选择器。代码采用 UI 与 Library 职责分离，业务服务通过接口注入，便于替换依赖测试。本文不提供未经测量的性能指标，也不宣称已完成大规模数据压力测试。

2 系统设计

2.1 架构设计

解决方案包含 ReadJournal、ReadJournal.Library 及 ReadJournal.UnitTest 三个工程。UI 负责视图、应用资源、定位器、导航实现、文件选择和提示；Library 负责模型、规则、存储、统计、备份及 ViewModel，不引用 Avalonia 控件；测试工程使用 xUnit、Moq 和临时真实 SQLite 数据库。

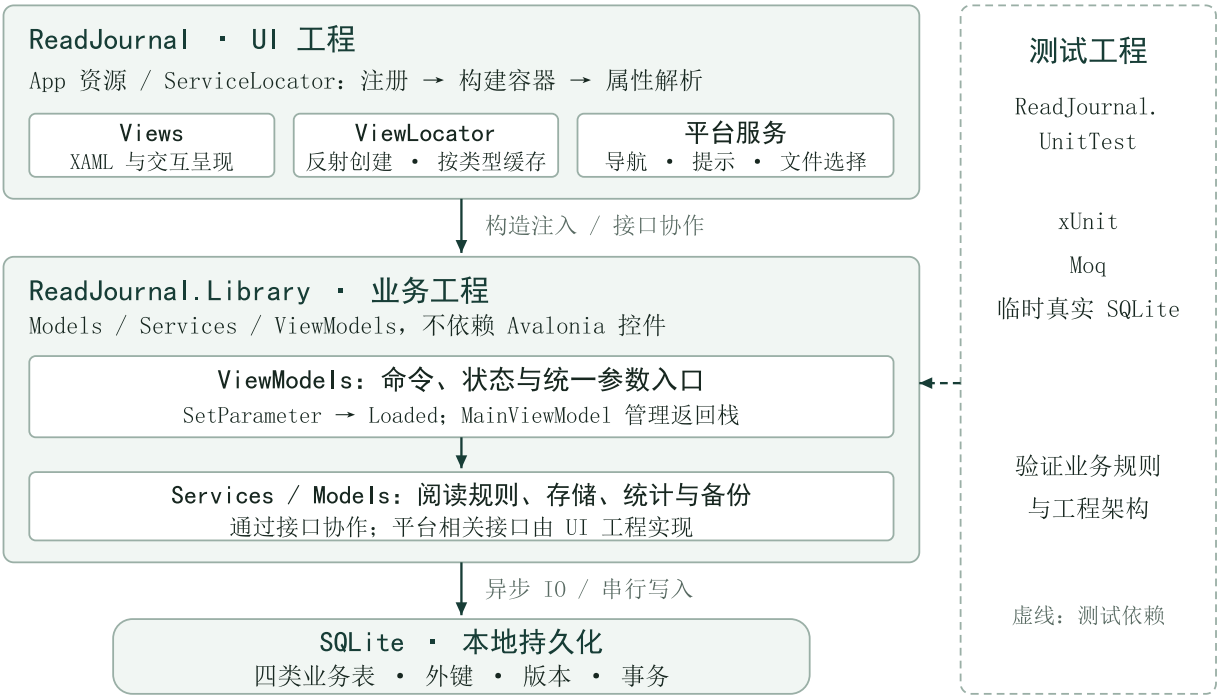


图 1　依据实际工程组织绘制的架构图

保留 DailyPoetryA 的完整组装机制：ServiceLocator 构造内局部 ServiceCollection 注册、BuildServiceProvider 构建、只读 IServiceProvider 保存、属性 GetService 解析；Current 通过 Application 资源读取并缓存。全部服务和 ViewModel 为单例，构造函数只保存依赖并建立命令，不启动导航或加载。

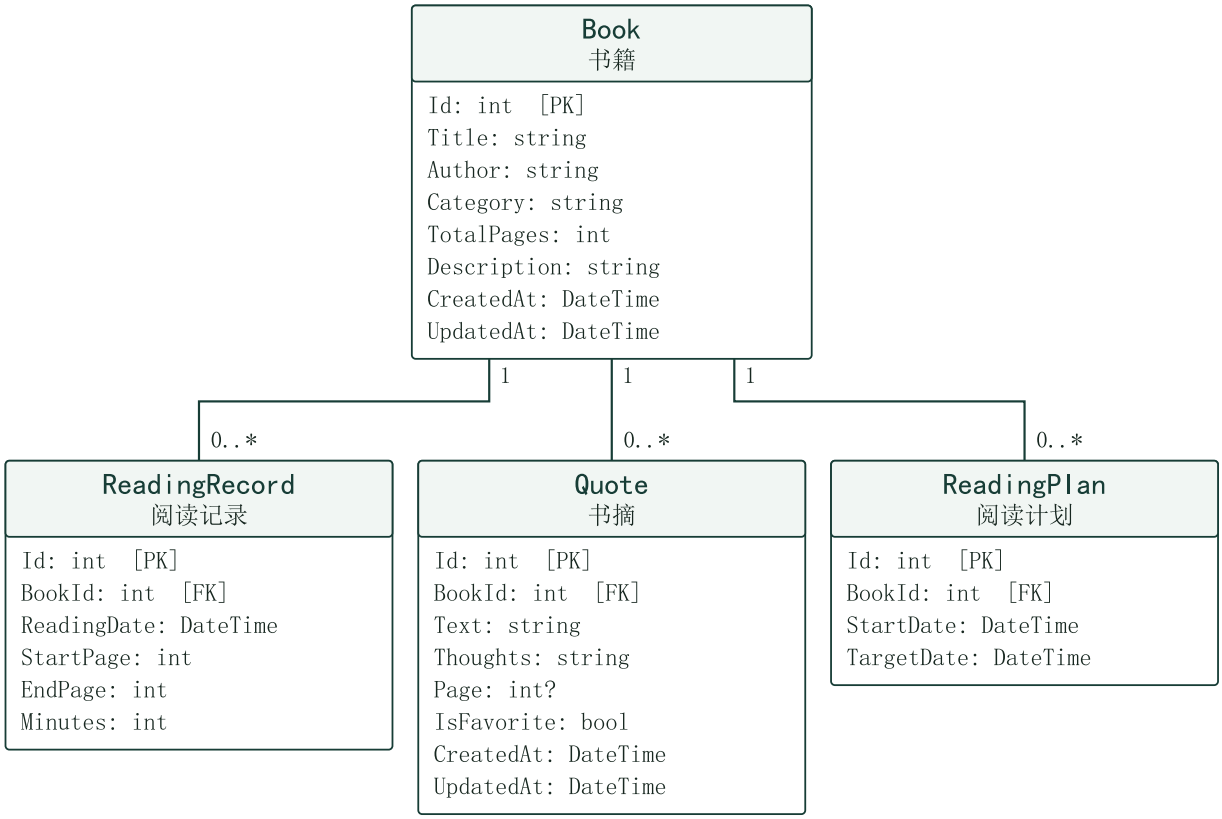
根导航替换 MainWindowViewModel.Content；菜单导航通过 SetMenuAndContent 清空返回栈并设置菜单；内容导航调用 PushContent 压栈。返回栈及菜单状态属于 MainViewModel，未增加导航宿主或页面工厂。ViewLocator 实现 IDataTemplate，以命名替换、反射和 Activator 创建视图，按类型缓存并设置 DataContext。

2.2 功能模块设计

菜单提供概览、书库、书摘、计划、统计及设置六个入口。书库进入详情或书籍编辑；详情进入书籍编辑、书摘编辑、计划编辑及记录弹窗；书摘和计划列表进入相应编辑页。内容返回保留上一级业务上下文，菜单切换清空历史栈。左上角返回仅在当前模块有上一页时显示，模块首页隐藏入口；编辑页仍执行未保存确认与忙碌保护。

导航顺序为定位目标单例、SetParameter、切换内容或压栈、Loaded 公开异步命令加载。加载失败保留目标页面，呈现错误及重试入口。重复同一单例内容目标在 SetParameter 之前拦截，避免覆盖栈中原参数；不同书籍经返回书库再打开，不构造同类型详情的多份历史快照。

2.3 数据/数据库设计



PK：主键；FK：BookId → Book.Id；删除书籍时级联删除子记录

图 2　实际四表实体关系；子表 BookId 外键级联删除

Book 含 Id、Title、Author、Category、TotalPages、Description、CreatedAt 和 UpdatedAt。ReadingRecord 含 Id、BookId、ReadingDate、StartPage、EndPage 和 Minutes。Quote 含 Id、BookId、Text、Thoughts、可空 Page、IsFavorite 及创建/修改时间。ReadingPlan 含 Id、BookId、StartDate 和 TargetDate。数据库 user_version 为 1，初始化事务创建表并启用外键。

记录约束为 1 ≤ 起始页 ≤ 结束页 ≤ 总页数、分钟数为正整数、日期不晚于本地当天。当前页取最大结束页，无记录为 0；累计页数对每条记录求闭区间长度之和，包含重读。完成日期为读至末页记录的最早日期。修改记录或总页数后由有效数据重新推导状态。

新建或编辑计划时，同一本书不能再关联第二个计划；普通新计划不能关联已完成书籍。旧备份中的多个完成计划可保留，但修改记录或总页数会使其同时变为未完成时，整个操作拒绝并提示先清理多余计划。未逾期的建议每日页数为剩余页数除以包含当天的剩余天数后向上取整；完成为 0；逾期单列状态与剩余页数。删除计划不删除记录。

2.4 接口与类设计

最终采用与参考相同的语法树口径：只扫描 UI 与 Library 手写 class/interface，包括默认访问级别；partial 按工程及完整类型名去重；排除测试和生成代码，不计枚举、结构体或 record。实际为 66 类、14 接口、77 个 C#源文件、14 个视图文件，其中业务视图 10 个。参考为 63 类、10 接口、9 视图；最低 45 类、7 接口，实际数量已满足。

14 项接口职责为数据库初始化、书籍存储、阅读记录存储、书摘存储、计划存储、偏好、阅读业务、统计、备份、提示、文件选择及三层导航。职责与实现逐一对应，未通过空接口或人为拆分补足数量。主要类还包括四类实体、进度/计划/统计投影、共享编辑基类、各页 ViewModel、平台视图和定位器。新增计划一致性规则类在事务中复用业务状态校验，避免不同写入路径与备份规则分歧。

完整逐类型名称、路径、行号及源文件哈希见 docs/application-type-inventory.json 和同名 Markdown。参考复核结果保留于 docs/reference-baseline-cloud.json，可用 docs/tools/TypeInventory 重跑；原 PowerShell 脚本仍保留。

2.5 技术选型说明

技术	实际版本和用途
.NET / C#	net8.0，SDK 8.0.424 固定，跨平台桌面业务。
Avalonia	11.0.10；Desktop、Fluent、Inter 同版本；行为包 11.0.10.9。
CommunityToolkit.Mvvm	8.2.2；通知属性、RelayCommand 及 AsyncRelayCommand。
sqlite-net-pcl	1.8.116；本地 SQLite 及事务。
Microsoft.Extensions. DependencyInjection	8.0.0；真实 IoC 容器和构造注入。
xUnit / Moq	2.4.2 / 4.20.70；测试 SDK17.6.0，Runner2.4.5。
Avalonia.Headless	11.0.10，仅用于自动化中的真实应用资源和视图机制测试。

版本遵循参考项目，不以升级替代原有架构。历史 Linux 桌面验证在 Xvfb 中启动 Avalonia 应用；本次另在 Windows 真实桌面完成业务闭环。Headless 断言与真实桌面操作分别记录。首次完整构建有 3 条 NETSDK1206 提示，源于固定 SQLite 原生依赖含旧 Alpine RID；Linux x64 与 Windows x64 实测正常，不据此宣称支持 Alpine。

3 系统实现

3.1 关键模块的核心实现

以下摘录直接来自交付源码。定位器构造中的注册均使用 AddSingleton；末尾构建真实容器，属性由容器解析。

```
public ServiceLocator()
{
    var serviceCollection = new ServiceCollection();
    serviceCollection.AddSingleton<DataPaths>();
    serviceCollection.AddSingleton<IDatabaseService, DatabaseService>();
    serviceCollection.AddSingleton<IRootNavigationService, RootNavigationService>();
    serviceCollection.AddSingleton<IMenuNavigationService, MenuNavigationService>();
    serviceCollection.AddSingleton<IContentNavigationService, ContentNavigationService>();
    serviceCollection.AddSingleton<MainWindowViewModel>();
    serviceCollection.AddSingleton<InitializationViewModel>();
    serviceCollection.AddSingleton<MainViewModel>();
    serviceCollection.AddSingleton<OverviewViewModel>();
    serviceCollection.AddSingleton<BookListViewModel>();
    serviceCollection.AddSingleton<BookDetailViewModel>();
    serviceCollection.AddSingleton<IReadingService, ReadingService>();
    serviceCollection.AddSingleton<IBookStorage, BookStorage>();

    serviceCollection.AddSingleton<ReadingPlanListViewModel>();
    serviceCollection.AddSingleton<ReadingPlanEditViewModel>();
    serviceCollection.AddSingleton<StatisticsViewModel>();
    serviceCollection.AddSingleton<SettingsViewModel>();
    _serviceProvider = serviceCollection.BuildServiceProvider();
}
public MainWindowViewModel MainWindowViewModel => _serviceProvider.GetService<MainWindowViewModel>()!;
public InitializationViewModel InitializationViewModel => _serviceProvider.GetService<InitializationViewModel>()!;
public MainViewModel MainViewModel => _serviceProvider.GetService<MainViewModel>()!;
public OverviewViewModel OverviewViewModel => _serviceProvider.GetService<OverviewViewModel>()!;
```

内容导航保留常量 switch 映射与定位器调用。在阅记新增的忙碌页面和重复单例保护之后统一注入参数并压栈，未引入其他导航机制。

```
    public void NavigateTo(string view, object? parameter = null)
    {
        ViewModelBase target = view switch
        {
            ContentNavigationConstant.BookDetailView => ServiceLocator.Current.BookDetailViewModel,
            ContentNavigationConstant.BookEditView => ServiceLocator.Current.BookEditViewModel,
            ContentNavigationConstant.QuoteEditView => ServiceLocator.Current.QuoteEditViewModel,
            ContentNavigationConstant.ReadingPlanEditView => ServiceLocator.Current.ReadingPlanEditViewModel,
            _ => throw new ArgumentException("未知或尚未实现的内容目标。", nameof(view))
        };
        var main = ServiceLocator.Current.MainViewModel;
        // Direct Add/Edit commands must obey the same busy-page boundary as menu/back/close.
        // Check before SetParameter so a rejected navigation cannot erase a cached editor's draft.
        if (main.Content?.IsBusy == true || main.ContentStack.Contains(target)) return;
        target.SetParameter(parameter!);
        main.PushContent(target);
    }
}
```

阅读业务通过有效记录计算当前进度、累计活动和完成日期，重读不会叠加当前进度。

```
    }
    public BookProgress Calculate(Book book, IEnumerable<ReadingRecord> records)
    {
        var all = records.Where(x => x.BookId == book.Id).ToArray();
        return new BookProgress
        {
            Book = book,
            CurrentPage = all.Select(x => x.EndPage).DefaultIfEmpty(0).Max(),
            TotalReadPages = all.Sum(x => (long)x.EndPage - x.StartPage + 1),
            TotalMinutes = all.Sum(x => (long)x.Minutes),
            CompletedOn = all.Where(x => x.EndPage == book.TotalPages).Select(x => (DateTime?)x.ReadingDate.Date).Min()
        };
    }
}
```

备份由格式版本和四类集合组成，每个持久化字段要求显式存在。恢复先验证完整文档，在串行事务边界捕获原快照并成功写出自动备份，再删除并恢复业务表。自动备份失败不替换，写入失败事务回滚，成功后用户再进入页面会重新加载；主题在独立偏好文件中。

3.2 核心功能与 UI 截图

以下十图保留 2026 年 10 月 20 日 Linux 真实 X11 窗口的功能展示，显示的是主动载入的虚构演示资料；它们属于历史版本，首页仍显示返回按钮，不代表本版入口状态，也不替代 Windows 验收。随后图 13～20 保留 Windows 业务验收证据，图 21 为本版源码独立解包首次启动，图 22～23 补充返回入口修复。非全部内容能在同一窗口高度内展示时，页面提供滚动；截图不补画隐藏内容。



图 3　阅读概览：全量指标、在读书籍和近期记录



图 4　我的书库：搜索、状态筛选、分页与详情入口



图 5　书籍详情：进度、阅读活动与计划关联



图 6　书籍编辑：独立业务表单



图 7　书摘列表：虚构摘录、感想和收藏入口



图 8　书摘编辑：正文、感想、页码和收藏



图 9　计划列表：派生状态与建议页数



图 10　计划编辑：实际关联书籍与目标日期



图 11　统计：191 页、210 分钟、完成 1 本及近七日趋势



图 12　设置与数据：主题、导出、恢复和演示入口

本次 Windows 本机界面另行新建虚构测试书：120 页，阅读 10～20 页、25 分钟，摘录第 15 页并制定当天到期计划。累计活动为 11 页、25 分钟，当前进度 20/120 页，剩余 100 页；与此前 Linux 演示库的 191 页等数据属于不同测试集，不混用统计。

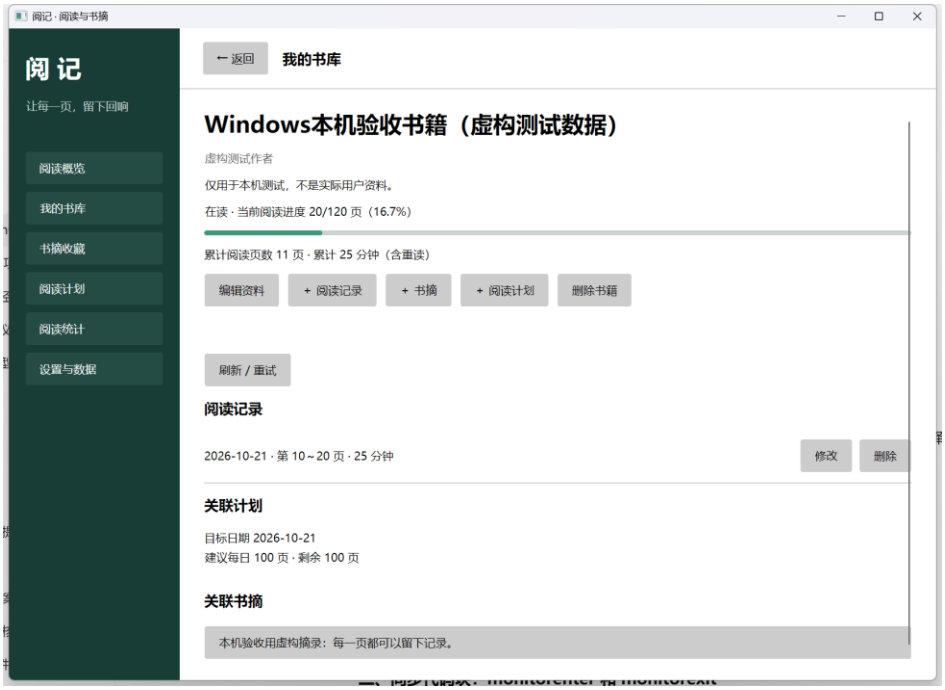


图 13　Windows 本机：阅读记录、书摘与当天到期计划的关联详情



图 14　Windows 本机：书摘收藏后显示星号，正文与感想保持



图 15 Windows 本机：实际统计 11 页、25 分钟、0 本完成



图 16 Windows 本机：删除后恢复并正常重启，业务数据与深色主题保留

3.3 难点和解决方案

实际问题	已执行处理及验证
参考源码最初不可用	Gitee 代理 CONNECT403，本地无参考目录；用户上传后同步副本，64 文件哈希及语法树类型清单与原基线完全一致，才进入骨架开发。
命令绑定和目标错误	实际参考 InitializationViewModel.cs:25 命令为 private，视图通过公开绑定访问；TodayViewModel.cs:76 用内容服务传菜单目标。阅记公开命令、按层使用服务与常量，参考副本保持只读。
XAML 与 SQLite 首轮失败	内部 xmlns 导致 AXN0002，移至根节点；busy_timeout 查询返回行，改用 ExecuteScalar。修复后重跑当前阶段构建及测试。
单例异步旧数据	引入参数版本校验，旧请求不发布到新参数页面；加载命令发现版本变化继续加载新参数。延迟任务用例验证 A 不会覆盖 B。
多次离开确认	主 ViewModel 共用防重入保护；返回、菜单、关闭不会并发打开多个确认窗口。保存失败保留草稿。
绑定诊断	事件命令显式绑定同一资源定位器中的单例，解决构造时 DataContext 未就绪；进度条使用静态模板，避免固定 Fluent 版本动画绑定。最终遍历十页绑定诊断为 0。
原生选择器驱动	GTK 路径自动补全曾干扰测试输入，应用正确提示文件不存在。通过真实列表选择同一文件后恢复成功，未用后台写库代替 UI 恢复。

本轮复核还发现加载与保存期间输入保护不足，以及计划状态变化与备份校验不一致。修复后，表单按字段合并未改值并保留用户修改，保存仅确认实际提交的快照；保存中新增修改保留在编辑页，新建成功回填主键避免重复插入。加载失败后可在编辑页重新加载。计划变更、记录修改与总页数修改在同一数据库事务中维护约束，拒绝不安全操作且不删除原数据。

后续修复还覆盖旧删除结果影响新导航、忙碌 Loaded 请求丢失、恢复期间导航，以及收藏与恢复交错。删除使用固定目标及参数版本检查；加载等待前次操作；所有页面忙碌时禁止菜单、返回、关闭和直接内容跳转。收藏由存储在同一事务内仅更新收藏位及时间，避免旧实体整体回写覆盖恢复数据。上述回归已纳入当前 94 项测试。本版另修复栈底仍显示无效返回入口的问题：CanGoBack 从主 VM 现有栈派生，集合变化同步按钮可见性和命令可执行状态；新增真实 MainView 绑定回归，保留菜单清栈与内容逐级返回。

重复目标拦截、参数版本、草稿保护、串行写入和事务均为阅记新增业务正确性措施。参考 README 明确未解决线程与原子性，不能把这些新增设计称为参考项目原有机制。

4 系统测试

4.1 测试环境

2026 年 10 月 21 日 Windows 本机实测使用 Windows x64（系统版本 10.0.26300）、.NET SDK 8.0.424、运行时 8.0.30 及 Avalonia 11.0.10。业务测试使用隔离临时 SQLite 数据库，架构测试使用真实应用资源和 Microsoft DI 容器。真实桌面通过 Windows UI Automation 操作控件，并结合鼠标键盘与进程窗口截图；未用 Headless 运行冒充 Windows 桌面。

保留历史平台证据：2026 年 10 月 20 日在 Debian 13 / Linux x64、Xvfb 21.1.16 / 1440×1000 环境完成原 68 项测试与桌面流程，后续收藏竞态修复版完成 93 项。历史数字仅对应当时版本；本版返回入口修复后的自动化测试为 94 项，验证平台为 Windows，不冒称已重新执行全部 Linux 流程，也不将多个平台的同一测试相加。

Windows 当前工作区完整 94 项通过；本次 v1 源码包在新目录独立解包，执行锁定还原、Release 构建和完整测试，同样 94 通过、0 失败、0 跳过。独立构建 0 错误、3 条 NETSDK1206 提示，来自固定 SQLite 依赖旧 Alpine RID。还原使用本机现有 NuGet 配置与缓存，未隔离全部缓存，不证明首次还原可离线完成。

```
dotnet restore ReadJournal.sln --locked-mode
dotnet build ReadJournal.sln -c Release --no-restore
dotnet test ReadJournal.sln -c Release --no-build --no-restore
```

正式交付统一放在 output/v1/。配套源码为 20261021-阅记-源码-v1.zip，共 119 个白名单文件、114395 字节，低于 10MB；无 bin、obj、编译结果或运行数据库。SHA256 为 be5d57de1e6001122a2299b9269e1aca0873bd9a134a53a4e83afbf2a0229187。本轮验证 CRC 正常、清单一致、所有文件与当前 Code 逐字节一致。

本次 v1 为统一交付版本，包含最新返回入口修复、对应回归和同步后的 README，不沿用旧版包的测试结果代替本轮独立验证。包内源码已独立完成还原、构建和 94 项测试；解包应用在另一个空数据目录实际启动，初始化版本 1 数据库并正常关闭。所有编译结果和运行数据仅留过程验证目录，未回填源码 ZIP。

本次对当前 Windows Release 程序集重新运行独立架构探针，71 项断言全部通过，覆盖 14 个 VM 单例、14 个服务接口、13 个反射视图、资源 Current、真实容器、12 个路由常量、参数、重复目标及返回栈。该 71 项与 94 项自动化测试分别统计，不合并为业务单元测试数量。

本版独立解包应用在隔离空库启动、初始化和正常关闭成功，启动日志未见绑定错误；返回修复另完成书库→详情→编辑→返回详情→返回书库的 Windows 实际点击。此前 Windows 完整业务闭环与 Linux 历史证据继续按原范围保留；本轮不声称重新遍历所有历史 UI 边界，未执行 Alpine、大数据压力及全部 CPU/系统组合。

4.2 测试用例

下表按本次独立解包 TRX 中的测试类归组统计，每条数据行对应实际执行用例；总计 94 项，全部通过。架构组新增返回入口与真实 MainView 命令绑定回归，其他组保留原有覆盖。

用例组与数量	主要实际覆盖	结果
架构 10	真实容器、单例、构造注入、三层导航、反射缓存、失败重试、A/B 切换、4 个忙态路由，以及返回入口与实际 MainView 绑定。	通过
数据库 3	初始化与持久化、外键事务回滚、初始化失败后重试。	通过
阅读业务 12	闭区间页数、重读、当前进度、末页完成、非法输入、搜索分页及关联删除。	通过
编辑 2	保存、放弃、取消离开及保存失败保留。	通过
扩展业务 16	计划边界、统计、演示、无效备份、JSON 往返、自动备份与恢复事务失败。	通过
导航时序 2	延迟旧请求不覆盖新参数；离开确认不并发重复。	通过
编辑回归 20	加载中草稿、首次加载门禁、失败重试、保存快照及新建 ID 回填。	通过
计划一致性 8	重复关联、历史多个完成计划的状态变化、原子拒绝、安全编辑及备份往返。	通过
删除导航竞态 6	固定删除目标、旧操作不误删新目标、不弹走新的页面或草稿。	通过
忙碌加载 2	单例重新打开时等待旧操作并加载最新参数。	通过
设置导航 4	数据操作期间菜单、返回与关闭保护，完成后恢复。	通过
概览架构 3	最近记录查询由存储提供，VM 职责及结果正确。	通过
恢复并发 6	收藏/恢复次序、忙态菜单返回关闭、原子收藏与失败释放保护。	通过

工作区原始结果为 docs/evidence/back-navigation-20261021/after-fix.trx；本轮独立源码结果为 docs/evidence/release-v1-20261021/clean-v1.trx，配有 restore.log、build.log 和 test.log。71 项独立架构结果为同目录 architecture/runtime-checks.json。所有数量依据实际记录，不以截图估算，也不把单次耗时作为性能保证。

Windows 历史业务验收完成 120 页书籍、非法页码校验、10～20 页/25 分钟记录、第 15 页书摘、计划与收藏，核对统计后进行 JSON 导出、级联删除、恢复和深色重启。删除后四表全 0，恢复后各 1 且逐字段一致，收藏保留；自动备份记录替换前空库，完整性正常、无外键错误。

4.3 测试截图

图 17～20 保留 Windows 实际业务验收截图，图 21 更新为本次 v1 源码包独立解包后的首次启动，图 22～23 为返回入口修复的实际窗口。94 项通过的数量依据上节两份 TRX 及本轮 test.log，71 项依据独立探针；不使用旧截图冒充最新测试终端结果。

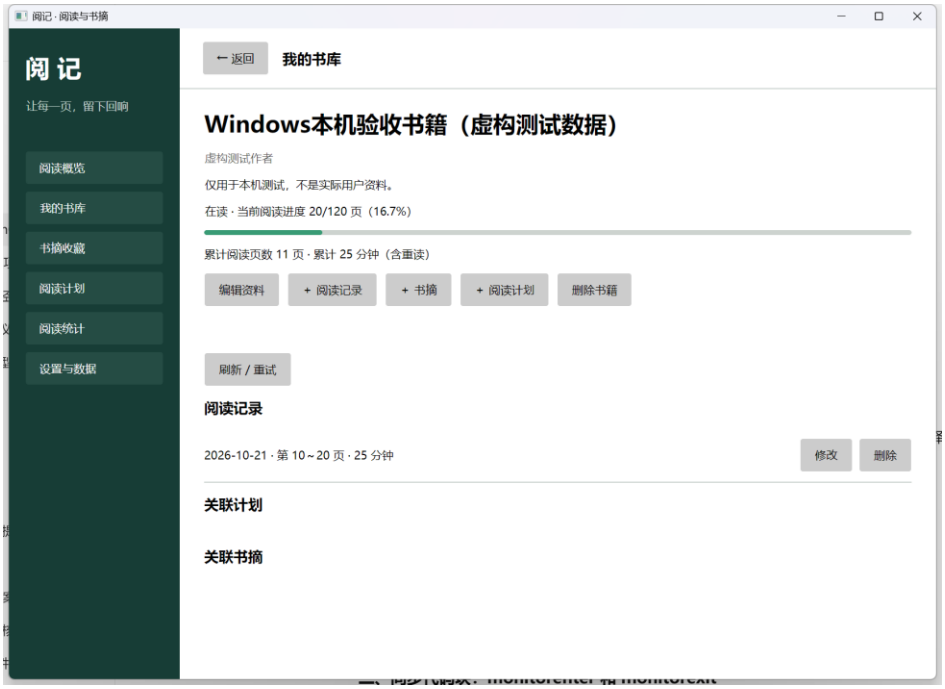


图 17　Windows 真实阅读记录保存：10～20 页按 11 页累计，进度 20/120 页

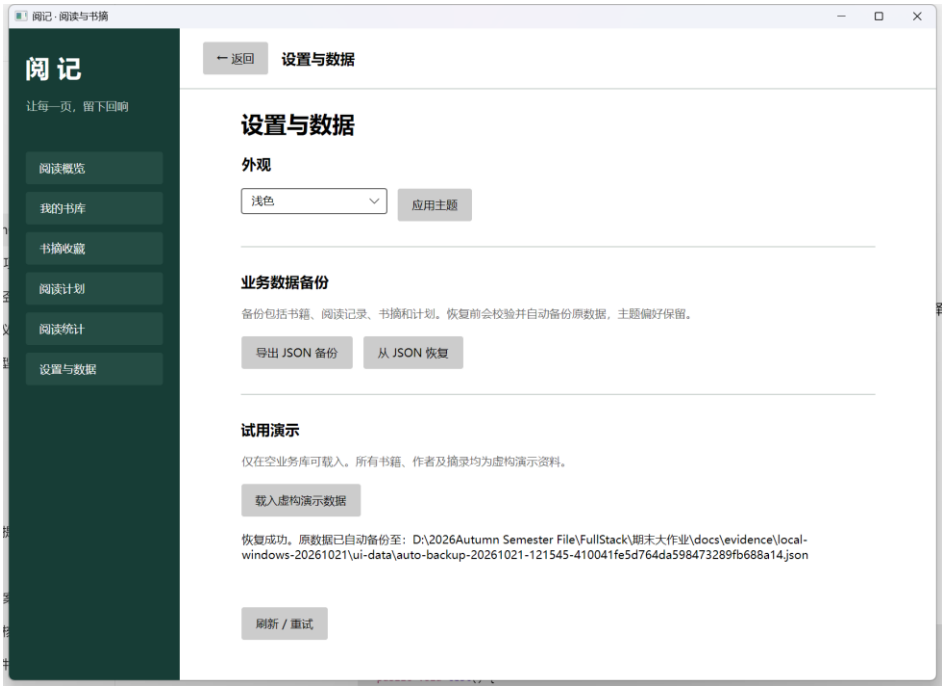


图 18　Windows 真实备份恢复成功，并显示原数据自动备份路径

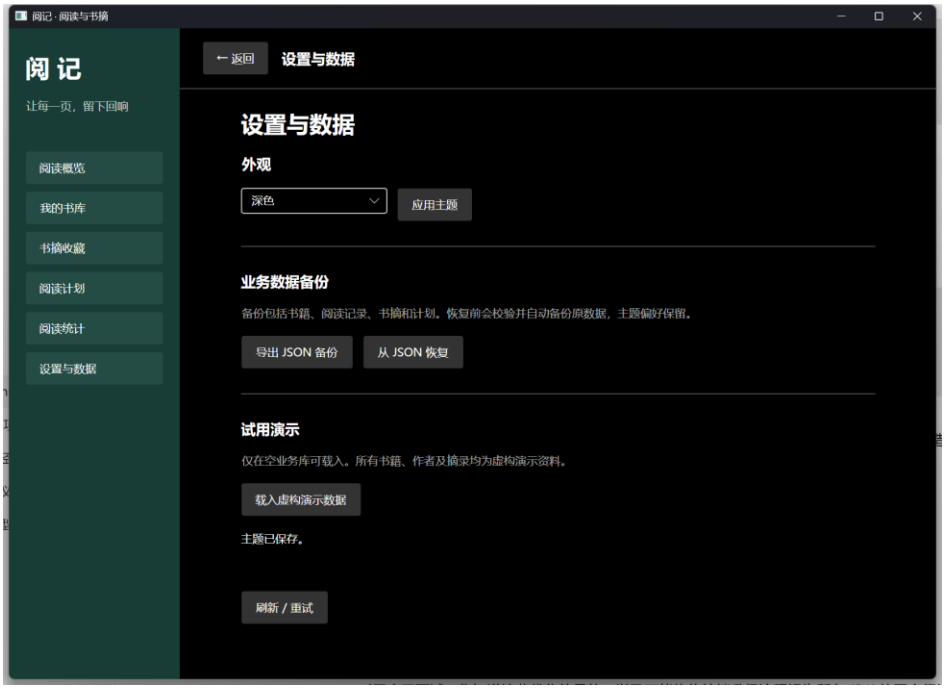


图 19 Windows 实际应用深色主题，偏好保存成功



图 20 Windows 正常关闭后重启：11 页、25 分钟及关联数据保留



图 21　v1 源码独立解包后的首次启动：隔离空库初始化成功

前述业务截图来自明确标注的虚构测试书，不是实际用户阅读资料。历史 Windows 逐字段核对结果保留于原验收证据；本版首次启动与返回验证使用另外的隔离数据目录，测试集不混用。本报告未对实际界面补画控件。

本次返回入口验证使用独立虚构书籍：从书库进入详情及编辑后逐级返回，页面栈保留；回到模块首页后不再展示无效入口。

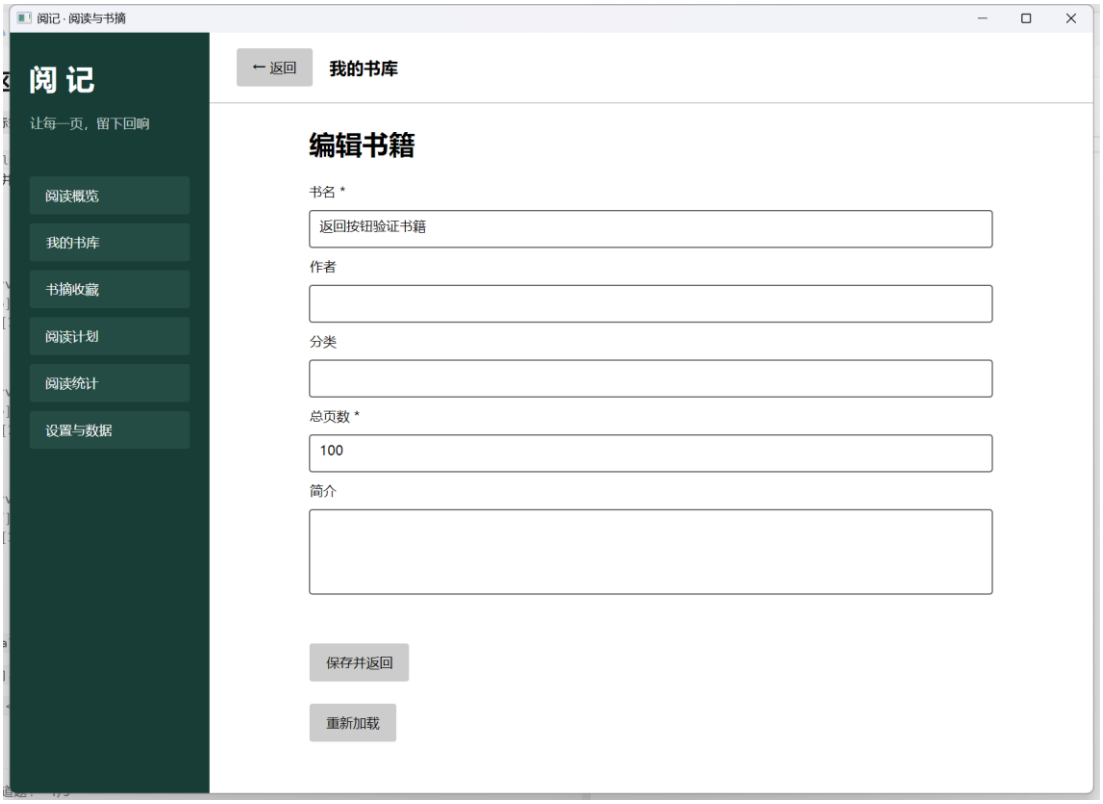


图 22　Windows 返回修复：编辑资料页有上一页时显示返回入口

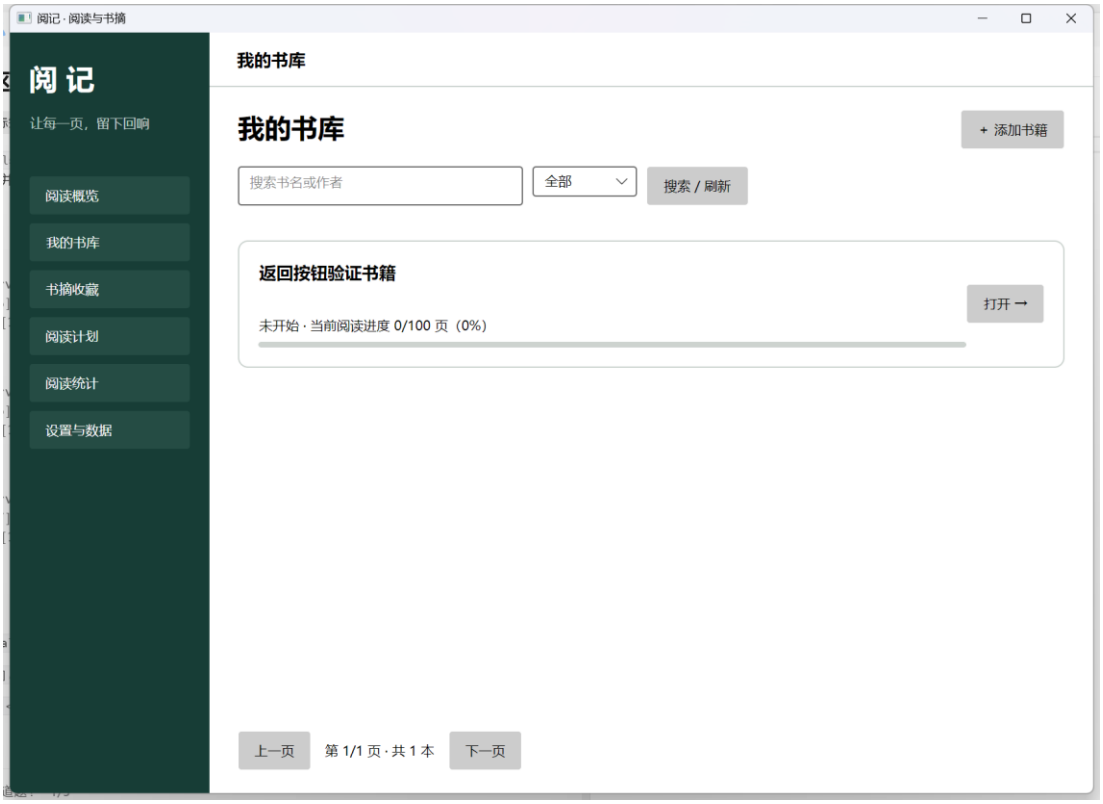


图 23　逐级返回书库首页后隐藏返回入口，测试书籍保留

5 总结与展望

阅记完成十个业务页面及阅读管理闭环，保留参考项目真实 IoC、单例定位器、三层导航、统一参数入口、主 ViewModel 返回栈和反射视图缓存。本版 Windows 工作区和 v1 源码独立解包均 94 项通过，当前程序集 71 项独立架构断言通过；源码包也已实际启动验证。新增返回入口状态修复完成真实点击验证，既有业务闭环和 Linux 结果保留其原验证范围。

本统一交付版 v1 已同步最新返回入口、94 项测试、独立源码验证和包元数据，交付目录含可编辑 Word、正式 PDF 和源码 ZIP；旧交付文件按用户要求清理。Windows 通过结论来自实际验证。本机构建仍有固定 SQLite 依赖的 Alpine RID 提示，未执行 Alpine 兼容、大数据压力或多设备同步测试。后续可继续改善输入可用性并测量更大数据集；电子书阅读、联网检索和云同步不在本版范围内。

交付与验证项目	本版状态
规模与架构	66 类、14 接口、10 业务页，八项机制对应；71 项独立运行时架构断言通过。
应用与业务测试	Windows 工作区与本次 v1 源码独立解包均 94 项通过；原有业务闭环及新增多级返回已实际验证。
数据安全与恢复	删除后四表全空；恢复后四表各 1 且逐字段一致；自动备份、完整性及外键核对正常。
源码与报告配对	统一 v1：Word、PDF 及源码 ZIP；源码 119 文件、114395 字节，逐文件一致。
继续改进的边界	后续可增加更大数据集测量、更多系统/CPU 组合及输入可用性验证；目前不宣称全部场景穷尽通过。